FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of FRIDAY 17 JANUARY 2025 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- **The exam is open book: textbooks and personal notes are permitted.**
- **Pencil writing is permitted, except on the cover page (this sheet).**
- **Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.**
- **Time:** lab part 1 h – **theory part 2 h.**

SCORING AND GRADE ATTRIBUTION:

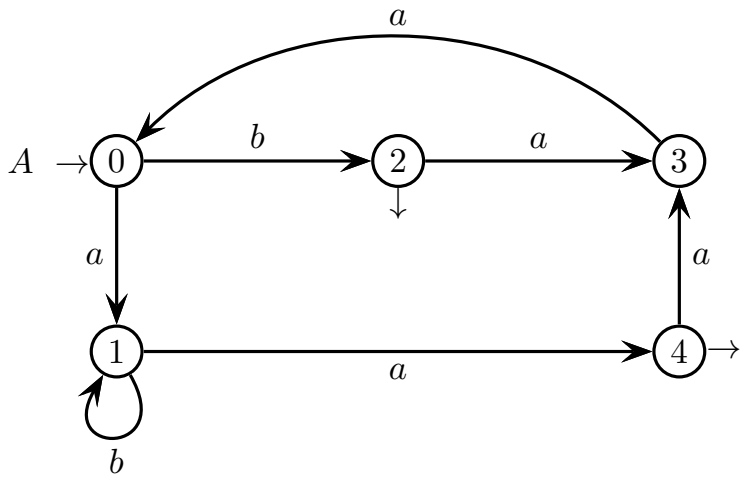
- The exam is in written form and consists of two parts:

Theory	syntax and semantics of languages, divided in four sections:	(75% of the final grade)
1)	regular expressions and finite automata	(1 exercise × 10 points)
2)	syntax analysis and parsing methodologies	(1 exercise × 10 points)
3)	grammar design – translation – semantic analysis	(2 exercises × 5 points)
4)	*bonus* question	(1 exercise × 5 points)
Lab	compiler design by Flex and Bison	(25% of the final grade)
1)	basic questions	(30 points)
2)	*bonus* question	(5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

1. Consider the following deterministic finite-state automaton A, with five states, over the two-letter alphabet $\Sigma = \{a, b\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Is automaton A minimal? If not, minimize it.

(b) By using the Brzozowski-McCluskey method (BMC), find a regular expression R_A that generates the language $L(A)$ of automaton A.

(c) Now consider the regular expression R below, over the same two-letter alphabet $\Sigma = \{a, b\}$:

$$R = \big([\, a \,]\, b\, a\, a \big)^* \qquad \text{// “[ ]” is the optionality operator}$$

Write all the strings of length four or less (≤ 4) of the language $L(R)$ generated by R.

(d) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A_R that recognizes language $L(R)$.

(e) Find the finite-state automaton that recognizes the mirror language $L(R)^R$. Is this automaton deterministic?

---

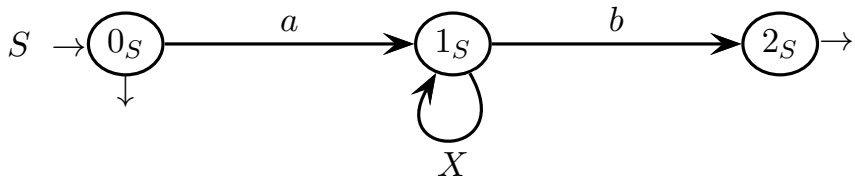
space for questions (a-d)

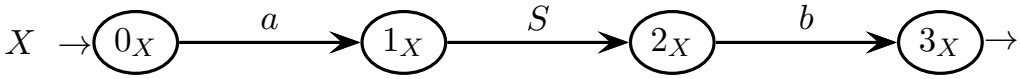
space for questions (a-d)

space for questions (a-d)

2 Syntax analysis and parsing methodologies (10 points)

1. Consider the EBNF grammar G below, represented as a network of recursive machines, over a two-letter terminal alphabet $\Sigma = \{a, b\}$ and a two-symbol nonterminal alphabet $V = \{S, X\}$ (axiom S):





Answer the following questions (use the figures / tables / spaces on the next pages):

(a) List all the strings of length six or less (≤ 6) of language $L(G)$ and draw their syntax trees.

(b) Draw the complete pilot of grammar G and prove that grammar G is of type ELR.

(c) Compute all the guide sets of grammar G and prove that grammar G is of type ELL, for some $k \geq 1$.

(d) Write the syntactic procedure of nonterminal S (axiom).

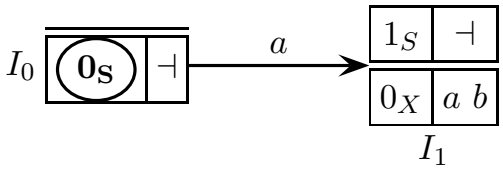
(e) Say if language $L(G)$ is regular and reasonably explain your answer.

(f) Suppose nonterminal X is nullable, e.g., by assuming that the net state 0_X is final (besides being initial). Orderly examine whether the new grammar G is ELL, ELR, or ambiguous. Explain your answers.

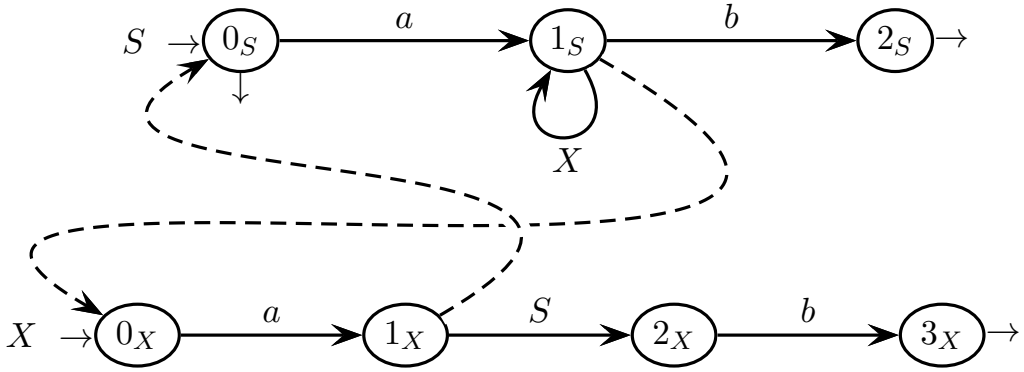
---

valid strings of length ≤ 6 and their syntax trees – question (a)

complete pilot of grammar G and conflict analysis – question (b)



guide sets of grammar G and conflict analysis – question (c)



syntactic procedure of nonterminal S (axiom) – question (d)

space for questions (e-f-g)

3 Grammar design – translation – semantic analysis ($5 + 5$ points)

1. Consider a language for describing a list of *three-dimensional meshes*. A mesh consists of a set of points in three-dimensional spaces, which are the *vertices* for the *triangles* constituting the surface of the mesh. The vertices appear as a list of points, and each vertex is given a unique zero-based numeric ID derived from the vertex position in the list. Thus, the first vertex has an ID of zero, the second vertex has ID 1, the third has ID 2, and so on. These IDs are used as references to the vertices in the description of each triangle.

   An example of a string in this language is the following:

```
mesh Pyramid
  vertices  (0,0,0) (2,0,0) (1,0,2) (1,2,1)
  triangles (0,1,2) (0,1,3) (1,2,3) (0,2,3)
end

mesh Triangle
  vertices  (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
end
```

   In a valid string, there have to be one or more *mesh descriptions*, enclosed within the `mesh` and `end` keywords. Each mesh has a *name*, which is an arbitrary identifier that appears immediately after the `mesh` keyword. Inside each description, there are two *sections*, for `vertices` and `triangles` respectively, introduced by eponymous keywords. Each section contains a non-empty list of three-element integer tuples (hence the coordinates of the vertices are also integers).

   Answer the following questions:

   (a) Write an EBNF non-ambiguous grammar that generates the language just described. In the grammar, use the terminals num and id (not to be further expanded) for integer numbers and arbitrary alphanumeric strings, respectively.

   (b) Draw the syntax tree of the sample string shown below:

```
mesh Triangle
  vertices  (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
end
```

   (c) Extend the grammar, so as that a mesh may optionally contain sub-meshes, as follows:

```
mesh Triangle
  vertices  (0,0,0) (1,0,0) (0,1,0)
  triangles (0,1,2)
  submesh   mesh X vertices (0,0,0) triangles (0,0,0) end
            mesh Y vertices (0,1,0) triangles (0,0,0) end
end
```

   You may just list the modified or new rules.

---

space for answering questions (a-c)

2. Consider the abstract syntactic support G shown below, over the terminal alphabet $\Sigma = \{ i, n \}$ and nonterminal alphabet $V = \{ S, A, B \}$, defined by these rules (axiom S):

$$G \begin{cases} 1: & S \rightarrow A\ B \\ 2: & A \rightarrow i\ A \\ 3: & A \rightarrow i \\ 4: & B \rightarrow n\ B \\ 5: & B \rightarrow n \end{cases}$$

The nonterminal A is a list of variable names, the nonterminal B is a list of numeric variable references, the terminal i represents an identifier, and the terminal n represents an integer.

We want to define an attribute grammar over the support G to perform a validity check of the variable references in B with respect to the amount of variables actually declared in A. For instance, if there are 5 variable declarations, and B contains a numeric reference greater than or equal to 5, then that reference is invalid. The same holds for any negative reference.

To this end, we define the following attributes:

type	*name*	*domain*	*(non)term.*	*meaning*
left	v	boolean	S, B	*valid*: true if a list of references (B) or the sentence (S) is valid
left	n	integer	A	*number*: the number of declarations in the list
right	m	integer	B	*maximum*: one past the maximum allowed variable reference

Answer the following questions (use the tables / trees / spaces on the next pages):

(a) Write the semantic functions for the three attributes v, n and m above.

(b) Decorate the syntax tree of the string:

`i i i 0 5 1 2`

with the value of the attributes. Use the syntax tree prepared on the next page.

(c) Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers.

(d) Consider a modified grammar where rule 1 is rewritten as: $S \rightarrow B\ A$. Explain if the new grammar still satisfies the L condition and reasonably justify your answers.

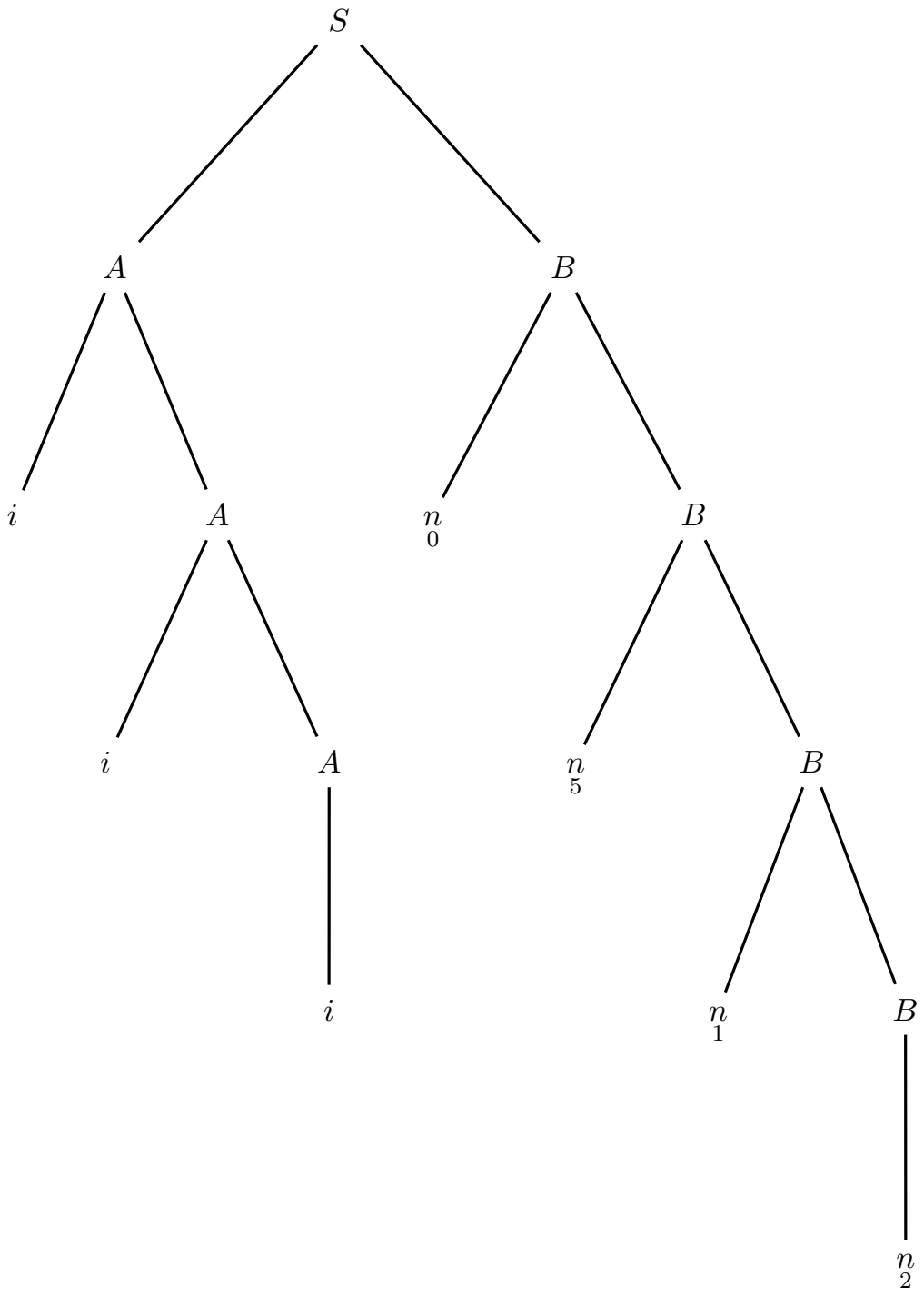
---

attribute grammar to write - question (a)

(for clarity the terminals are also indexed)

#	*syntax*	*semantics*
1:	$S_0 \rightarrow A_1\ B_2$	
2:	$A_0 \rightarrow i_1\ A_2$	
3:	$A_0 \rightarrow i_1$	
4:	$B_0 \rightarrow n_1\ T_2$	
5:	$B_0 \rightarrow n_1$	

syntax tree to decorate – question (b)



space for answering questions (c, d)

4 Bonus question (5 points)

1. Consider the following short program written in pseudo-code, with three integer variables a, b and c:

```
do
    read(a)
    b := b + a
while a > 0
c := 0
while b > 0 do
    c := c + 1
    b := b − 5
write(c)
```

   Answer the following questions (use the figures / tables / spaces on the next pages):

   (a) Translate the program to a control flow graph, where each node contains at most one statement or one expression.

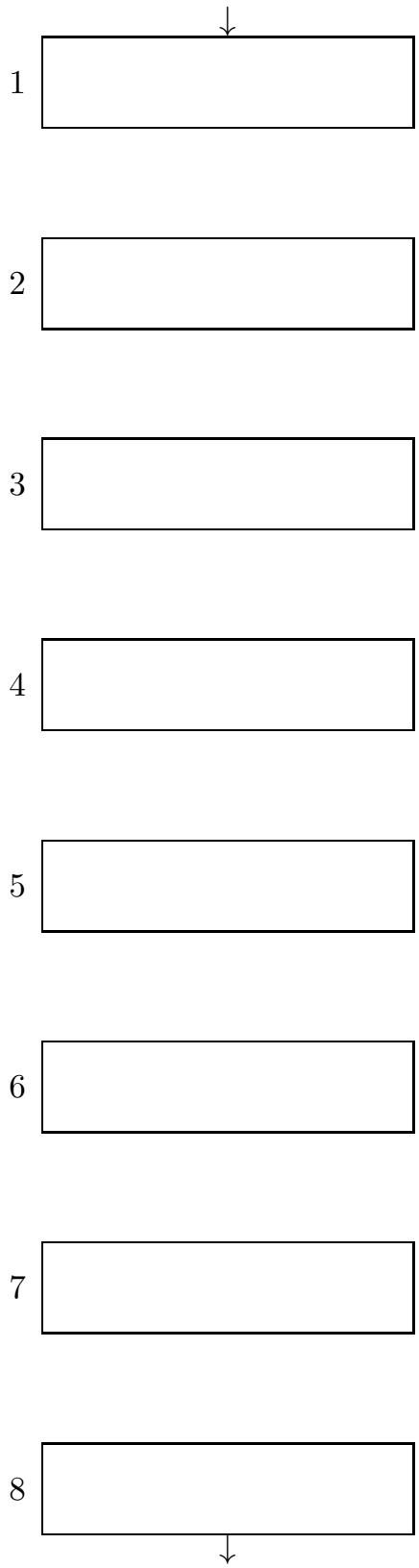
   (b) Annotate the nodes of the control flow graph with the liveness of the variables a, b and c, using your intuition.

   (c) Write the definition of the equations for computing the liveness *in* and *out* of the node corresponding to the condition check $b > 0$, including the values of the *use* and *def* sets for that node.

   (d) Are there any variables that are used without being initialized? How can this be understood from the liveness information?

---

control flow graph to be completed for questions (a, b)



space for answering questions (c, d)

free space – any questions

free space – any questions